

编译器设计专题实验报告

实验五：语义分析框架实现

张凌曜 2234214014 计算机 2306

1 实验目的

在词法分析和语法分析的基础上，构建 AST（抽象语法树）和符号表，做类型检查和语义错误检测（比如重复声明、未声明变量）。

2 实验原理

语义分析是编译的第三阶段。AST 用树形结构表示程序的语法层次，符号表记录变量名、类型、作用域这些信息，类型检查就是看赋值和运算中类型对不对得上。

本实验用递归下降解析器，解析的同时建树和填符号表。

3 实验过程

3.1 AST 节点

每个 AST 节点有类型（FuncDecl、VarDecl、Assign、If、While 等）、值（变量名、运算符）、数据类型（int/float）和子节点列表。

3.2 符号表

符号表用作用域栈实现。每进一个代码块（函数体、if 块）压一层新作用域，退出时弹出。变量查找从内到外逐层找，先找到先用，这样能处理变量遮蔽。重复声明检测只看当前作用域内有没有同名变量。

3.3 类型推断

二元运算的类型推断规则很简单：任一操作数是 float 结果就是 float，否则是 int。

4 测试结果

测试程序：

```
1 int main() {  
2     int x;  
3     float y;  
4     x = 5;  
5     y = 3.14;  
6     if (x < 10) {  
7         x = x + 1;  
8     }  
9     while (y > 0) {  
10        y = y - 1.0;  
11    }  
12    print x;  
13    return 0;  
14 }
```

程序输出的 AST（缩进表示层级）：

```
1 Program  
2   FuncDecl(main) :int  
3     Params  
4     Block  
5       VarDecl(x) :int  
6       VarDecl(y) :float  
7       Assign(x) :int  
8         ID(x)  
9         IntConst(5) :int  
10      Assign(y) :float  
11        ID(y)  
12        FloatConst(3.14) :float  
13      If  
14        BinOp(<)  
15          ID(x) :int  
16          IntConst(10) :int  
17        Block  
18          Assign(x) :int  
19            ID(x)  
20          BinOp(+) :int  
21            ID(x) :int  
22            IntConst(1) :int  
23      While  
24        BinOp(>)
```

```

25         ID(y) :float
26         IntConst(0) :int
27     Block
28         Assign(y) :float
29             ID(y)
30         BinOp(-) :float
31             ID(y) :float
32             FloatConst(1.0) :float
33     Print
34         ID(x) :int
35     Return
36         IntConst(0) :int

```

符号表在作用域 0 记录了函数 main 的签名 int()。语义检查结果：未检测到语义错误，符合预期——变量声明、赋值、if/while 的类型都匹配正确。

```

===== 抽象语法树 (AST) =====
Program
  FuncDecl(main) :int
    Params
    Block
      VarDecl(x) :int
      VarDecl(y) :float
      Assign(x) :int
        ID(x)
      IntConst(5) :int
      Assign(y) :float
        ID(y)
        FloatConst(3.14) :float
      If
        BinOp(<=)
        ID(x) :int
        IntConst(10) :int
        Block
          Assign(x) :int
            ID(x)
          BinOp(+) :int
            ID(x) :int
            IntConst(1) :int
        While
          BinOp(>)
          ID(y) :float
          IntConst(0) :int
          Block
            Assign(y) :float
              ID(y)
            BinOp(-) :float
              ID(y) :float
              FloatConst(1.0) :float
      Print
        ID(x) :int
      Return
        IntConst(0) :int
===== 符号表 =====
作用域 0:
  main : int()
===== 语义检查结果 =====
未检测到语义错误

```

请输入源文件路径：===== 源代码 =====

```

int main() {
    int x;
    float y;
    x = 5;
    y = 3.14;
    if (x < 10) {
        x = x + 1;
    }
    while (y > 0) {
        y = y - 1.0;
    }
    print x;
    return 0;
}

```

===== Token 流 =====

```

<INT, int> <ID, main> <LPA, (> <RPA, )> <LBR, {> <INT, int> <ID, x> <SCO, ;> <FLOAT, float> <ID, y> <SCO, ;> <ID, x> <ASG, => <NUM, 5> <SCO, ;> <ID, y> <ASG, => <FLO, 3.14> <SCO, ;> <IF, if> <LPA, (> <ID, x> <LSS, <> <NUM, 10> <RPA, )> <LBR, {> <ID, x> <ASG, => <ID, x> <ADD, +> <NUM, 1> <SCO, ;> <RBR, }> <WHILE, while> <LPA, (> <ID, y> <GTR, >> <NUM, 0> <RPA, )> <LBR, {> <ID, y> <ASG, => <ID, y> <SUB, -> <FLO, 1.0> <SCO, ;> <RBR, }> <PRINT, print> <ID, x> <SCO, ;> <RETURN, return> <NUM, 0> <SCO, ;> <RBR, }>

```

5 实验总结

递归下降写起来有个容易搞混的地方：同样是 `int x` 后面跟分号是变量声明，跟左括号就是函数声明，需要往前多看一个 Token 才能确定。

整体来说 AST 能比较清楚地反映程序结构，符号表的作用域栈处理变量遮蔽也没问题。

A 附录：代码仓库

本实验的全部源码、报告及 Web 演示平台已托管至 GitHub：

<https://github.com/yaossss123/compilation-web>